

Image Restoration for Rain and Snow with PromptIR

VRDL Homework 4

Hung-I Yang 112550097

June 1, 2026

1 Introduction

1.1 Task description

This homework is an *image restoration* task: given a degraded RGB image corrupted by **rain** or **snow**, predict the clean target. The training set has 3,200 aligned pairs (1,600 per degradation type) at 256×256 . The test set has 100 degraded images without type labels; a **single** model must restore both degradations. The competition metric is **PSNR**.

1.2 Core idea

I analyze rain vs. snow statistics on the training pairs (severity, gradients, sparsity) to motivate a **residual spatial gate** on decoder prompts, **wider features** (dim=64) for extreme rain, **L1+SSIM** [2] for structure fidelity, **eight-fold geometric TTA** at inference, and—in the final recipe—an **EMA** (exponential moving average) of the trainable weights with decay $\alpha=0.999$, updated every step and used together with TTA when exporting predictions. Training and inference code: <https://github.com/harry0816web/NYCU-VRDL/tree/main/hw4>.

1.3 Dataset statistics (motivation)

Rain is far harder than snow (approx. degraded-to-clean PSNR ~ 12.2 dB vs. ~ 22.9 dB; mean L1 residual ~ 55.9 vs. ~ 13.0 on $[0, 255]$ scale). Rain streaks are strongly vertical; snow is more isotropic and sparse ($\sim 44\%$ of pixels nearly clean). These facts motivate spatially selective prompting and capacity scaling rather than only tuning the learning rate.

2 Method

2.1 Data preprocessing and augmentation

Training pairs are normalized to $[0, 1]$. I randomly crop 128×128 patches (same offset on degraded and clean). Unless otherwise overridden at run time, **geometric augmentation** uses eight modes: identity, horizontal/vertical flips, and rotations by 90° , 180° , 270° , applied identically to both branches. A fraction of training data can be held out for validation. At test time the network sees full 256×256 images.

2.2 Model architecture

I use **PromptIR** [1]: a U-Net encoder–decoder with Transformer blocks and decoder-side **PromptGenBlock** modules. An overlap 3×3 stem maps RGB to base width $C=64$ with

channel stages $C, 2C, 4C, 8C$ and attention heads $(1, 2, 4, 8)$. Downsampling uses 3×3 convolution plus PixelUnshuffle(2); the bottleneck runs at 32×32 with width $8C$. Each **TransformerBlock** stacks **MDTA** (channel-wise transposed self-attention with depthwise 3×3 context and learnable temperature) and **GDFN** (gated depthwise feed-forward, expansion factor 2.66, $\text{GELU}(\text{DWConv}(x_1)) \odot \text{DWConv}(x_2)$). The block counts per level are $(4, 6, 6, 8)$; four refinement blocks sit at the finest resolution before a 3×3 head predicts a **residual** added to the input.

Residual spatial prompt. Prompt weights are still produced from global pooling, but I modulate the upsampled prompt with a lightweight spatial gate $G \in [0, 1]^{H \times W}$ (1×1 reduce, ReLU, 3×3 , sigmoid) using a **residual multiplier** so clean regions do not lose prompt information:

$$\text{Prompt}_{\text{spatial}} = \text{Prompt}_{\text{original}} \odot (1 + G). \quad (1)$$

2.3 Objective

The training loss combines pixel L1 with a structure term:

$$\mathcal{L} = \mathcal{L}_{\text{L1}} + \lambda_{\text{SSIM}}(1 - \text{SSIM}(\hat{y}, y)), \quad \lambda_{\text{SSIM}} = 0.1. \quad (2)$$

2.4 Inference: test-time augmentation (TTA)

At inference I apply **eight-fold geometric TTA** (test-time augmentation), also known as *self-ensemble*. The eight transforms correspond to the **dihedral group** D_4 —all symmetries of a square:

1. Identity (no transform),
2. Horizontal flip,
3. Vertical flip,
4. Rotation by 90° ,
5. Rotation by 90° + horizontal flip,
6. Rotation by 180° ,
7. Rotation by 270° ,
8. Rotation by 270° + horizontal flip.

For a single test image x , the procedure is:

$$\hat{y} = \frac{1}{8} \sum_{k=0}^7 T_k^{-1}(f_\theta(T_k(x))), \quad (3)$$

where T_k denotes the k -th geometric transform, T_k^{-1} its inverse, and f_θ the restoration network. By averaging predictions across all orientations, high-frequency artifacts that depend on spatial direction are suppressed, yielding a smoother and more robust output at zero extra training cost (only $8 \times$ inference FLOPs). In my experiments TTA consistently provides +0.25–0.30 dB improvement.

2.5 Hyperparameters and training strategy

Table 1 lists final training configuration.

Table 1: Training configuration:	
Field	Value
<i>Model</i>	
Input / output channels	3 / 3
Base width (dim)	64 (stages 64–128–256–512)
Blocks per level	(4, 6, 6, 8)
Refinement blocks	4
Attention heads	(1, 2, 4, 8)
FFN expansion	2.66
Use bias in convs	False
LayerNorm	with bias (WithBias)
Decoder prompts	enabled
Spatial residual prompt	True
<i>Optimization and schedule</i>	
Optimizer	AdamW
Learning rate	2×10^{-4}
Weight decay	10^{-4}
Warmup epochs	15 (linear)
Max epochs	80 (early exps); 150 (best entry)
Cosine $\eta_{\min}$	10^{-6}
Batch size	8
Patch size	128×128
Data loader workers	4
Precision	16-bit mixed
<i>Loss and stabilization</i>	
Loss	L1+SSIM
SSIM term weight λ_{SSIM}	0.1
EMA	enabled, decay $\alpha=0.999$, from epoch 0
<i>Data augmentation and validation</i>	
Geometric augmentation	enabled (eight D_4 modes)
Validation fraction	0.05
Validate every	1 epoch
<i>Run infrastructure</i>	
Random seed	42
GPUs	1

3 Results

3.1 Training & Validation Curves

3.1.1 Baseline (L1, dim=48, 80 epochs)

I start from the vanilla PromptIR architecture with base width 48, L1 loss, eight-fold geometric augmentation, and 128×128 random crops, trained for 80 epochs. This run serves as the reference point for all subsequent ablations, achieving 29.64 dB on the public test set.

Before changing the architecture or training recipe, I also evaluate **eight-fold TTA** on the *same* baseline checkpoint by using eight forward passes per test image and averaging after inverse transforms, which tends to suppress directional artifacts such as vertical rain streaks.

As can be seen in Table 2, the baseline checkpoint alone scores **29.64 dB** without TTA, while the same weights with eight-fold geometric TTA reach **29.94 dB**—a **+0.30 dB** gain solely from ensembling flipped/rotated views at test time.

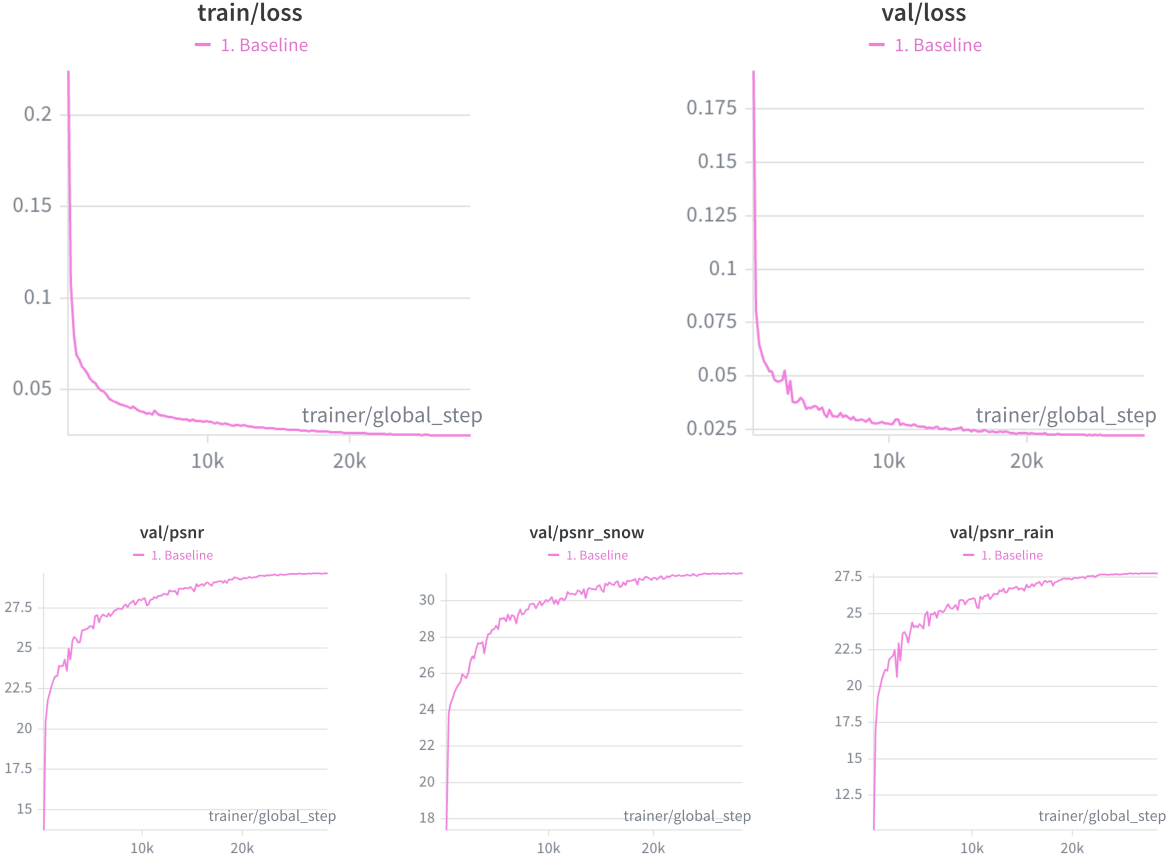


Figure 1: Baseline (dim=48, L1): training and validation loss (top); validation PSNR overall, snow-only, and rain-only splits (bottom).

Table 2: Public test PSNR: baseline training and TTA at inference (same weights).

Setting	PSNR (dB)	Δ
Baseline (L1, dim=48, 80 ep., no TTA)	29.64	—
+ 8-fold TTA	29.94	+0.30

3.1.2 Residual spatial prompt

The dataset analysis (Section 1.3) shows that snow is highly sparse ($\sim 44\%$ pixels nearly clean) while rain covers almost the entire image. The original PromptGenBlock uses global average pooling to produce prompt weights, so clean and degraded regions receive equal prompt intensity. A lightweight spatial gate should help the model focus restoration on degraded pixels while leaving clean regions untouched. I add the residual spatial gate ($1+G$) described in Section 2.2 on top of the baseline recipe (still dim=48, L1, 80 epochs), obtaining a clear +0.40 dB

improvement that validates spatially selective prompting for mixed-degradation restoration.

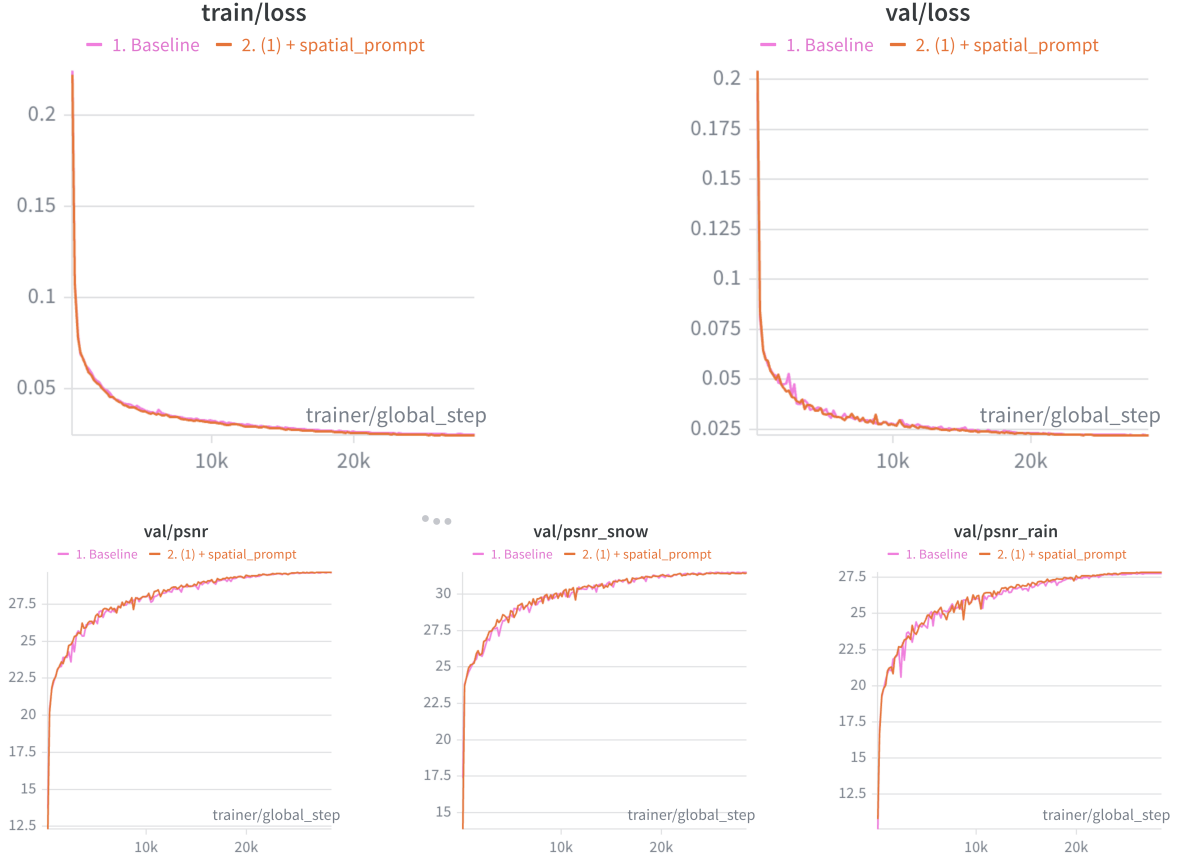


Figure 2: + residual spatial prompt (dim=48, L1): losses (top); validation PSNR overall, snow, rain (bottom).

Table 3: Public test PSNR: spatial prompt vs. baseline training.

Setting	PSNR (dB)	Δ
Previous (baseline)	29.64	—
+ residual spatial prompt	30.04	+0.40

3.1.3 Wider model (dim=64) and longer training

Inspecting the training curves of the 80-epoch baseline and spatial-prompt runs, I observe that both loss and PSNR have not fully plateaued—the model is still improving when training ends. Additionally, rain degradation is extremely severe, suggesting the dim=48 network may lack the capacity to simultaneously learn rain and snow restoration well. I therefore widen the model to dim=64 (~ 63 M params) and extend training to 150 epochs so the larger model has enough iterations to converge. This yields a massive +0.80 dB improvement (30.04 $\rightarrow$ 30.84, as can be seen in Table 4), the single largest gain in my entire experiment ladder. What’s more, in Figure 3, there is a clear gap in validation PSNR, confirming both the capacity bottleneck and the under-training of earlier runs.

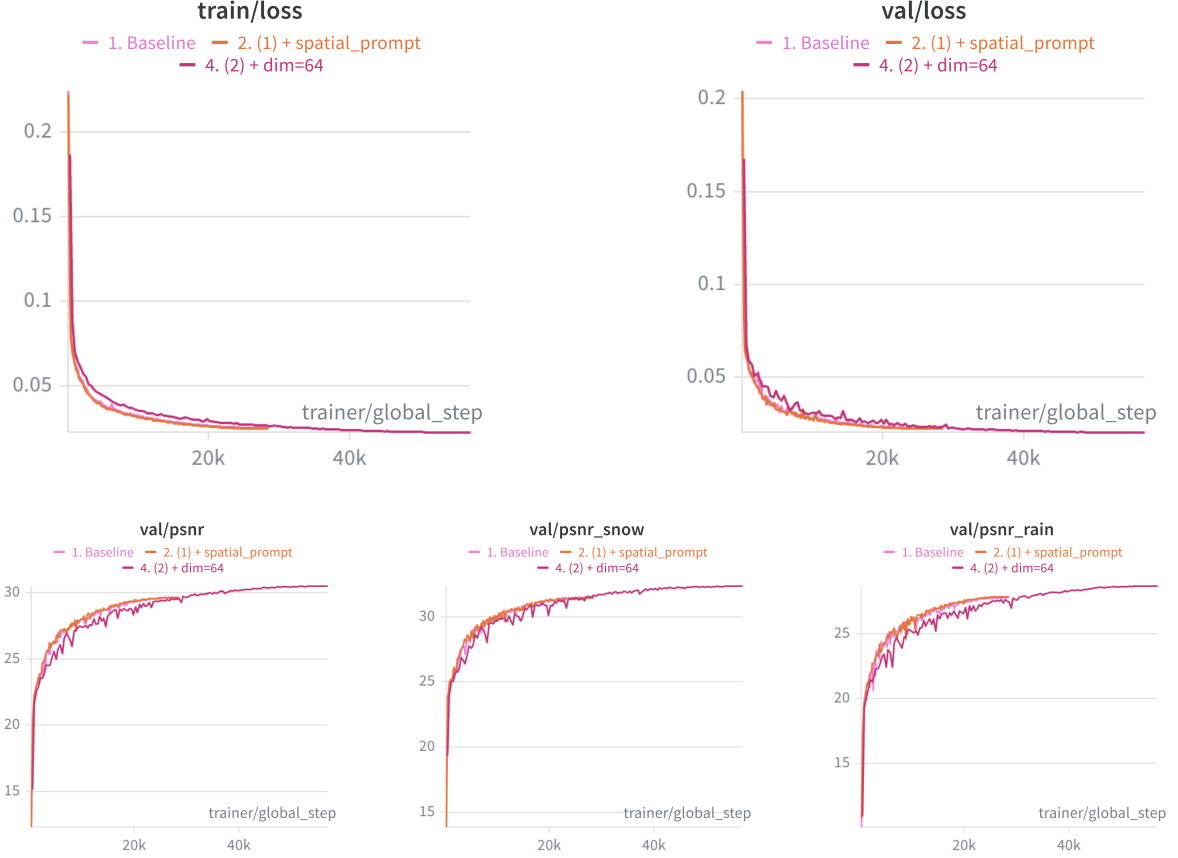


Figure 3: + dim=64, 150 epochs (parent = spatial prompt): losses (top); validation PSNR overall, snow, rain (bottom).

Table 4: Public test PSNR: dim=64, 150 epochs vs. spatial prompt.

Setting	PSNR (dB)	Δ
Previous (+ spatial prompt)	30.04	—
+ dim=64, 150 ep.	30.84	+0.80

3.1.4 Charbonnier loss (failed)

With architecture and schedule settled, I turn to the loss function. Charbonnier loss ($\sqrt{(x - y)^2 + \epsilon^2}$) is a smooth L1 variant commonly used in super-resolution; its near-zero gradient when the residual is small should prevent the model from over-correcting already-clean pixels. However, replacing L1 with Charbonnier ($\epsilon=10^{-3}$) actually drops PSNR by -0.28 dB (as can be seen in Table 5). I conjecture that for the extremely heavy rain degradation in this dataset, the sub-linear gradient of Charbonnier at large residuals slows down learning where aggressive correction is needed most.

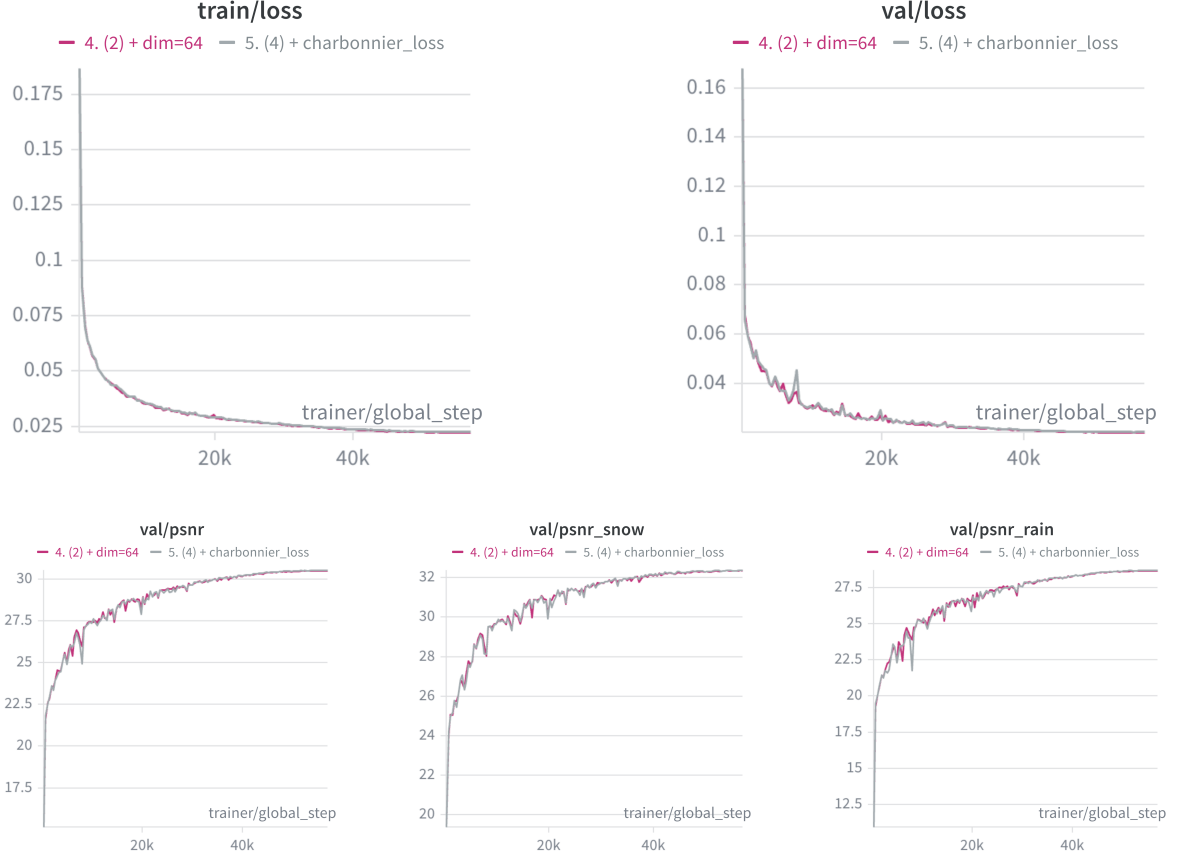


Figure 4: + Charbonnier loss (dim=64 parent): losses (top); validation PSNR overall, snow, rain (bottom).

Table 5: Public test PSNR: Charbonnier vs. L1 at dim=64.

Setting	PSNR (dB)	Δ
Previous (dim=64, L1)	30.84	—
+ Charbonnier	30.56	-0.28

3.1.5 L1+SSIM loss

Since Charbonnier failed, I try a different angle: adding a structural similarity (SSIM) term to L1. SSIM directly penalizes structural distortions (blur, contrast shifts) that L1 alone tends to miss, and may produce outputs that are more perceptually faithful without sacrificing pixel-level accuracy. I return to the dim=64 L1 parent and add $\lambda_{\text{SSIM}}=0.1 \cdot (1-\text{SSIM})$. Training remains identical aside from the loss. As can be seen, this recipe gain better validation PSNR than Charbonnier (Figure 5). What’s more, Table 6 shows that this recipe lifts public PSNR from the previous best **30.84 dB** to **31.07 dB**, a gain of **+0.23 dB**.

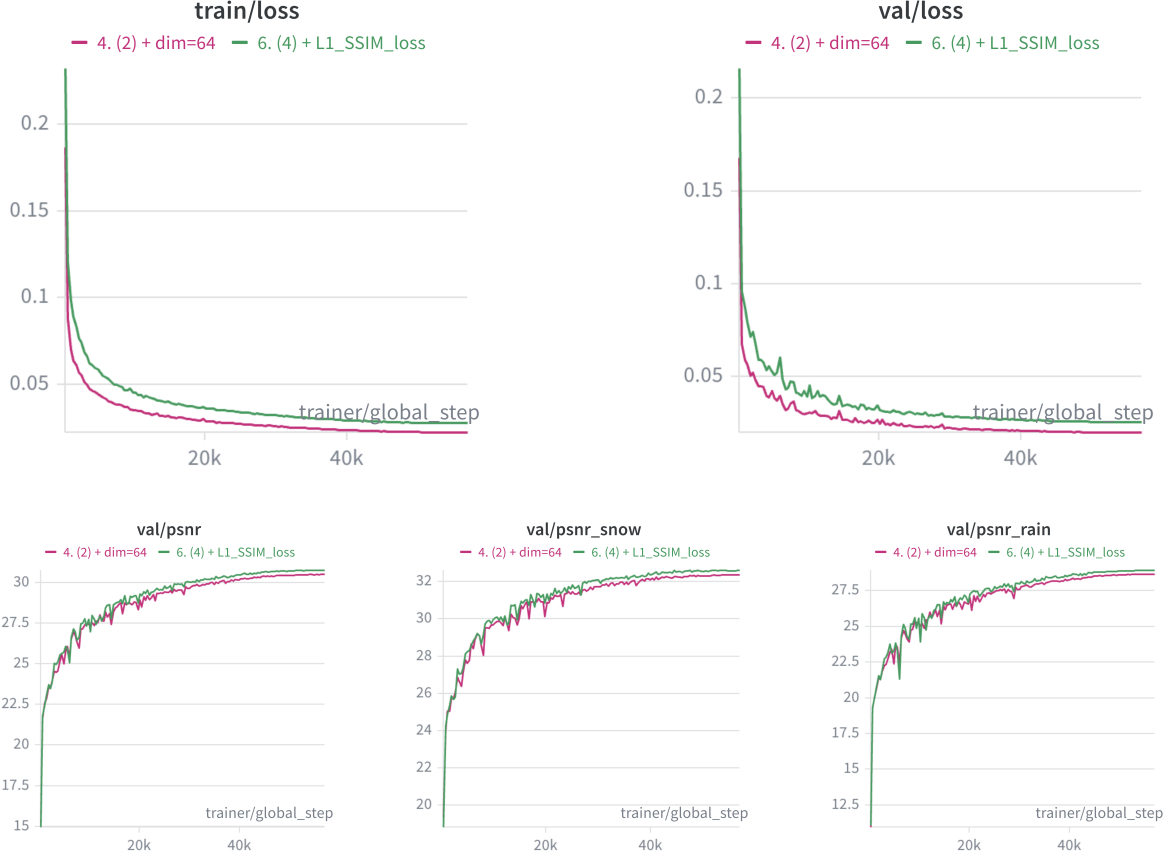


Figure 5: + L1+SSIM (dim=64 parent): losses (top); validation PSNR overall, snow, rain (bottom).

Table 6: Public test PSNR: L1+SSIM vs. previous best (dim=64 L1 parent, Section 3.1.3).

Setting	PSNR (dB)	Δ
Previous best (Section 3.1.3)	30.84	—
+ L1+SSIM	31.07	+0.23

3.1.6 CutMix (same degradation type)

With only 3,200 training pairs, the model may still benefit from stronger data augmentation beyond geometric flips. CutMix pastes a random rectangular region from another same-type sample (rain $\leftrightarrow$ rain or snow $\leftrightarrow$ snow) onto the current pair, forcing the model to handle degradation-boundary artifacts and increasing effective sample diversity. I stack CutMix on top of the L1+SSIM recipe (Section 3.1.5) and evaluate with eight-fold TTA. As can be seen in Figure 6, the CutMix run (orange) consistently achieves *higher* validation PSNR than the L1+SSIM parent (green) throughout training—particularly on rain samples, where CutMix provides a clear margin in the later epochs. Despite this validation advantage, the public test PSNR drops slightly to 31.02 dB (-0.05 vs. the parent’s 31.07 dB with TTA). A likely explanation is that CutMix introduces artificial rectangular degradation boundaries that never occur in real test images; the model spends capacity learning to handle these synthetic seams, which helps on the training-like validation split but does not transfer to the test set.

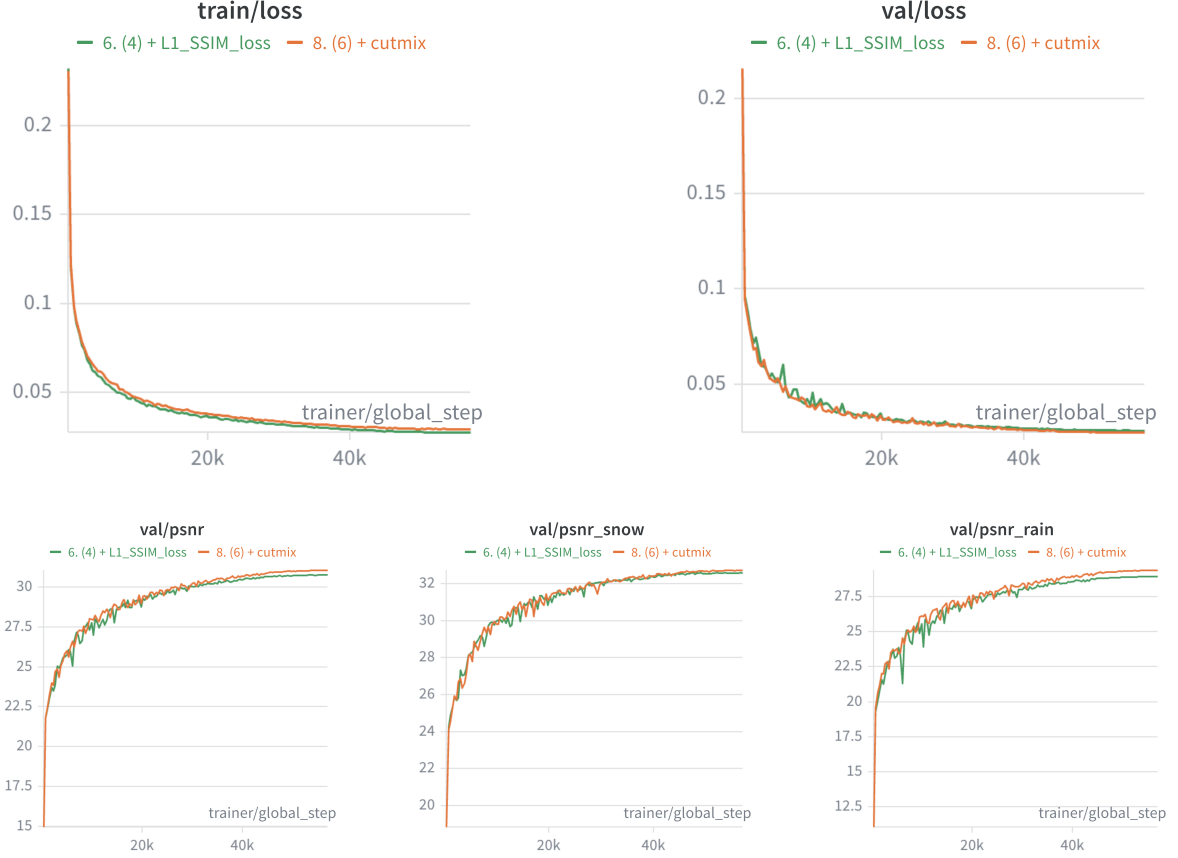


Figure 6: + CutMix (orange) vs. L1+SSIM parent (green): val loss and val PSNR (top); val PSNR by degradation type (bottom). CutMix achieves higher validation PSNR across the board, yet the test score is slightly lower.

Table 7: Public test PSNR: CutMix + TTA vs. L1+SSIM + TTA.

Setting	PSNR (dB)	Δ
Previous (L1+SSIM + TTA)	31.07	—
+ CutMix + TTA	31.02	−0.05

3.2 Roadmap summary

Table 8 repeats the full ladder for quick reference (aligned with the per-step tables; the L1+SSIM gain vs. the Section 3.1.3 best is summarized in Table 6).

Table 8: Cumulative public test PSNR summary (dB).

#	Main change	PSNR	Δ vs. parent
1	Baseline L1, dim=48, 80 ep.	29.64	—
1+TTA	+ TTA on (1)	29.94	+0.30
2	+ spatial prompt	30.04	+0.40
3	+ dim=64, 150 ep. (from 2)	30.84	+0.80
4	+ Charbonnier (from 3)	30.56	−0.28
5	+ L1+SSIM (from 3)	31.07	+0.23
6	+ CutMix (from 5)	31.02	−0.05
F	EMA (Add. Exp.; Sec. 4)	31.07	same as 5

4 Additional Experiments

4.1 Exponential moving average (EMA) of weights

Hypothesis. Mini-batch noise and aggressive augmentation cause the online weights $\theta^{(t)}$ to oscillate near the end of training. A slowly moving average θ_{EMA} should track a wider basin around good minima, making evaluation on the shadow weights less sensitive to which exact epoch is saved, even if the headline PSNR is similar to the last online checkpoint.

How this may (or may not) work. EMA maintains a *shadow copy* θ_{EMA} alongside the online parameters θ , updated after every gradient step:

$$\theta_{\text{EMA}}^{(t)} = \alpha \theta_{\text{EMA}}^{(t-1)} + (1 - \alpha) \theta^{(t)}, \quad \alpha = 0.999. \quad (4)$$

Because α is close to 1, θ_{EMA} effectively averages over $\sim 1/(1-\alpha)=1,000$ recent parameter snapshots, smoothing out stochastic oscillations. The shadow is initialized as a clone of $\theta^{(0)}$ and updated from step 0 onward. During validation, θ_{EMA} is temporarily swapped into the network to monitor EMA performance; after validation the online weights θ are restored so training continues unaffected. At deployment, only θ_{EMA} is loaded for inference. The overhead is one extra copy of the weight buffer in GPU memory (~ 250 MB for $\text{dim}=64$) and a trivial per-step update; no extra forward passes are needed during training. If late-training noise dominates, EMA should yield a smoother, more generalizable predictor; if the online optimizer already converges stably, θ_{EMA} may stay close to θ and leave public PSNR unchanged.

Result and analysis. Table 9 compares the L1+SSIM + $\text{dim}=64$ + spatial-prompt recipe under eight-fold TTA when weights are taken from the online checkpoint vs. from EMA. The public PSNR is **unchanged at 31.07 dB**: EMA does not beat the headline number on this leaderboard. Notably, as can be seen in Figure 7, the EMA run achieves noticeably higher validation PSNR than the L1+SSIM parent (Section 3.1.5) during the first half of training, suggesting that weight averaging effectively stabilizes the noisy early-stage optimization. However, both runs converge to similar validation PSNR by the end, and the test score remains the same. The practical benefit of EMA is therefore operational—it reduces variance across late-epoch checkpoints, making the final result less dependent on which exact epoch happens to be saved.

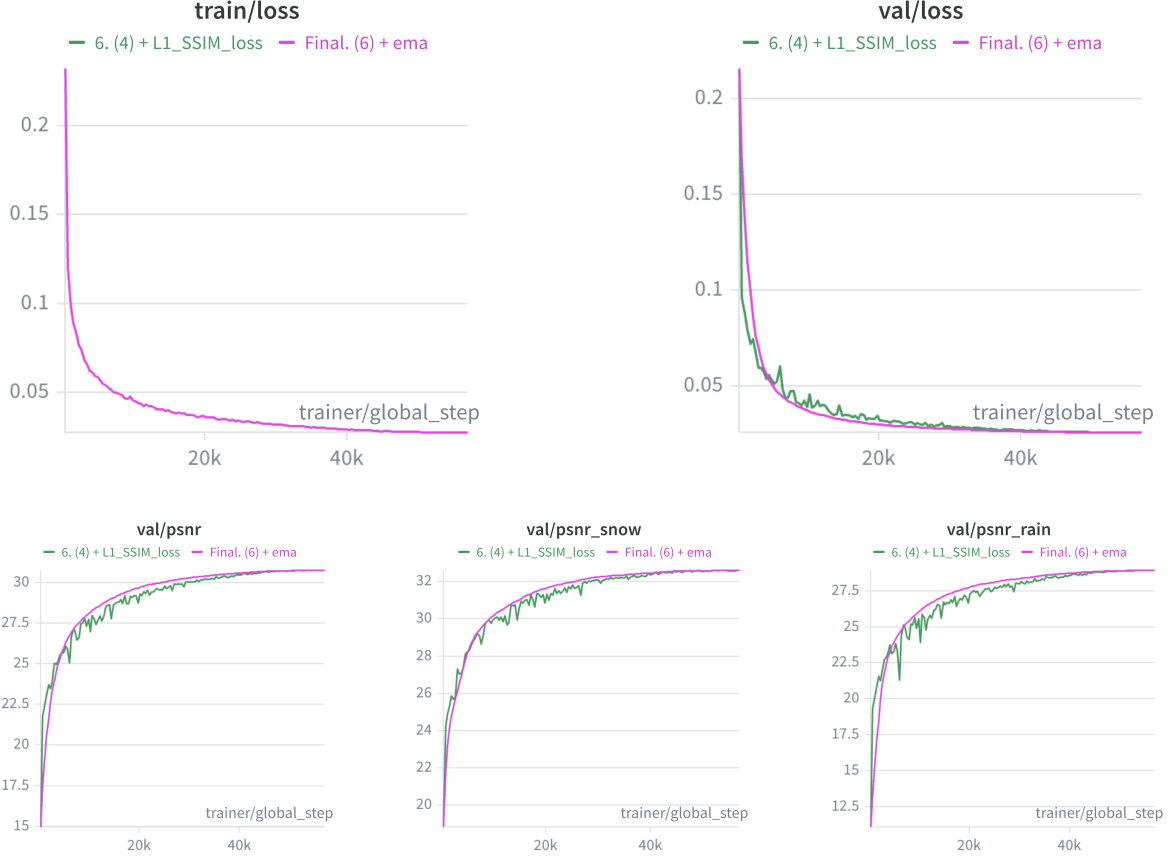


Figure 7: Training run with EMA enabled (same recipe as L1+SSIM): losses (top); validation PSNR overall, snow, rain (bottom).

Table 9: Public test PSNR: EMA vs. online weights (both with eight-fold TTA).

Setting	PSNR (dB)
L1+SSIM + dim=64 + spatial, online weights + TTA	31.07
Same recipe, EMA weights + TTA	31.07

5 Conclusion

I adopted PromptIR [1] with a residual spatial prompt, increased width to 64, L1+SSIM training [2], and eight-fold TTA, improving public PSNR from 29.64 dB to **31.07 dB**. The dominant lever was widening the network and training longer (+0.80 dB); the spatial prompt (+0.40 dB) and TTA (+0.25 dB) each contributed meaningfully. Charbonnier loss hurt performance, and CutMix improved validation metrics but slightly degraded test PSNR, likely due to artificial boundary artifacts absent in real test images. EMA (Section 4) matched the best score while stabilizing late-training checkpoints.

References

- [1] Vaishnav Potlapalli, Syed Waqas Zamir, Salman Khan, and Fahad Shahbaz Khan. PromptIR: Prompting for all-in-one blind image restoration. In *Advances in Neural Information*

Processing Systems (NeurIPS), 2023.

- [2] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.